

Model Prediction

1. Experiment Objective

Use a trained custom YOLO model to run inference predictions on new data, and verify the detection performance of the custom model in a ROS2 environment.

2. Experiment Principle

1. Glossary

Term	Description
YOLO	You Only Look Once, a real-time object detection algorithm that completes detection and localization in a single inference
Model Prediction	Using a trained model to run inference on new input data and output detection results
Custom Model	A model file (<code>best.pt</code>) trained by the user on their own dataset, optimized for a specific scenario
<code>best.pt</code>	The best weight file saved by Ultralytics YOLO after training, which contains the model structure and parameters
Inference	The process of feeding input data into a trained model and obtaining the prediction result

2. Technical Architecture

The workflow for ROS2 inference with a custom model is exactly the same as with a pretrained model — you only need to point the model path parameter to your custom `best.pt` file.

It supports 5 tasks × 3 input sources, with the corresponding launch files:

Task	Real-time Camera	Image	Video
Object Detection	<code>yolo_realtime_detect.launch.py</code>	<code>yolo_image_detect.launch.py</code>	<code>yolo_video_detect.launch.py</code>
Instance Segmentation	<code>yolo_seg_realtime.launch.py</code>	<code>yolo_seg_image.launch.py</code>	<code>yolo_seg_video.launch.py</code>
Pose Estimation	<code>yolo_pose_realtime.launch.py</code>	<code>yolo_pose_image.launch.py</code>	<code>yolo_pose_video.launch.py</code>
Image Classification	<code>yolo_cls_realtime.launch.py</code>	<code>yolo_cls_image.launch.py</code>	<code>yolo_cls_video.launch.py</code>
Oriented Object Detection	<code>yolo_obb_realtime.launch.py</code>	<code>yolo_obb_image.launch.py</code>	<code>yolo_obb_video.launch.py</code>

All launch files come from the `microros_v2_yolo_pkg` package, and the custom model path is specified via the `model_path` parameter.

3. Experiment Steps

1. Hardware Requirements

Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripheral	ESP32 camera (realtime mode)

2. Prerequisites

- A custom model `best.pt` has been trained (you can place your own trained model here)
- Put the model file in a path accessible to the YOLO package (e.g. `/home/yahboom/MODELS/yolo/`)
- Type `yolo_env` in the terminal and press Enter to enter the virtual environment

3. Image Mode (Custom Model Detection)

```
ros2 launch microros_v2_yolo_pkg yolo_image_detect.launch.py \
  model_path:=/home/yahboom/MODELS/yolo/yolo26n.pt
```

4. Video Mode (Custom Model Detection)

```
ros2 launch microros_v2_yolo_pkg yolo_video_detect.launch.py \
  model_path:=/home/yahboom/MODELS/yolo/yolo26n.pt
```

5. Real-time Camera Mode (Custom Model Detection)

Launch the proxy + camera + joint model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic.
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

Terminal 2 — Launch YOLO real-time detection (custom model):

```
# The model path can be replaced with a personally trained model
ros2 launch microros_v2_yolo_pkg yolo_realtime_detect.launch.py \
  model_path:=/home/yahboom/MODELS/yolo/yolo26n.pt
```

6. Other Task Examples

You can replace the model path in the commands below with your own trained model

Custom segmentation model real-time inference:

```
ros2 launch microros_v2_yolo_pkg yolo_seg_realtime.launch.py \
  model_path:=/home/yahboom/MODELS/yolo/yolo26n-seg.pt
```

Custom classification model image inference:

```
ros2 launch microros_v2_yolo_pkg yolo_cls_image.launch.py \
  model_path:=/home/yahboom/MODELS/yolo/yolo11n-cls.pt
```

7. How to Stop

Press `Ctrl+C` in each terminal, or press `q` or `ESC` to exit.

4. Experimental Phenomena

- Use the custom model to detect/segment/classify objects of custom classes.
- Higher accuracy than the pretrained model in specific scenarios.
- The detected object classes match the classes labeled in the training dataset.

5. Common Problems

Problem	Cause	Solution
Model fails to load	Wrong path or incompatible model format	Check the <code>best.pt</code> path and export it with ultralytics
Low detection accuracy	Insufficient training data or overfitting	Increase the amount of data, apply data augmentation, adjust epochs
Slow inference with the custom model	Model architecture too large	Use <code>yolo11n</code> as the base model when training and exporting
The <code>model_path</code> parameter has no effect	The default value in the launch file overrides it	Make sure to use the <code>:=</code> assignment syntax instead of <code>=</code>